Kiln Research Guide: Claude Code Mods and Cross-Session Dashboards

Researched 2026-10-12.

1. Reel Claims vs. Official Documentation

What the Reel Claims

In an October 2026 reel, creator Chris Ducktate demonstrated Duckboard (https://www.instagram.com/reel/DeCu7c8BZmd/), a custom mod inside Claude Code providing a unified dashboard of five concurrent sessions (MCP server, iOS, desktop, web). The reel claimed:

- Claude Mods allow arbitrary UI modifications and behavior customization.
- Duckboard displays live per-session context percentages, cache warmth counters, and rate limits (5-hour and weekly).
- Users can trigger one-click actions (`Compact`, `Retry`, `Details`) across idle or running sessions directly from a docked panel.
- Setup instructions and prompts are hosted at the creator's personal website.

What Official Documentation Confirms

According to the official Claude Code Overview (https://code.claude.com/docs/en/overview) and Claude Code Mods Overview (https://code.claude.com/docs/en/plugins/mods/overview):

- **Platform Architecture:** Mods run functions directly inside Claude Code (unlike external settings hooks or MCP servers). They require Claude Code v2.1.287+ in the terminal or v2.1.286+ in the Desktop app.
- **Plugin Structure:** Mods ship inside plugins containing `plugin.json` (manifest), `hooks.json` (hook registrations), and `register.js` (module registering handlers for events like `tool.call` and `ui.render`).
- **UI Capabilities:** Mods can draw panes beside the transcript, render banners above the prompt, restyle spinners, and add custom `/command` endpoints.
- **Reference Implementations:** Official sample mods hosted in `claude-code-playground` include `token-weather` (context window forecast), `blast-radius` (previews risky shell commands), and `replay-theater` (turn-by-turn edit replay).
- **Built-in Mods:** Several native features run as mods, including `/diff`, `agents-md`, `sec-default`, and `telemetry`.

What Remains Uncertain or Unverified

- **Cross-Session Native Discovery:** Official documentation details hooks running within an individual Claude Code session instance. Official docs do not describe a built-in cross-session IPC API. A mod monitoring external concurrent sessions must read external shared directories or socket state written by other background agent processes.
- **Duckboard Source Availability:** The exact implementation shown in the reel is proprietary/unverified and was not published in an open repository.

- **Desktop Docking Consistency:** While UI rendering works on Terminal and Desktop surfaces, web and headless cloud sessions do not support custom drawing.

---

2. Security Boundaries and Operational Limits

- **No Sandboxing:** Official documentation explicitly warns that mods run with full user account permissions. A mod can execute system binaries, read environment variables (including `ANTHROPIC_API_KEY`), rewrite prompts, intercept tool inputs, or approve tool executions without user interaction.
- **Prompt Injection & Data Exfiltration:** Installing third-party mods from untrusted marketplaces introduces supply chain risks identical to running untrusted Node.js/shell scripts.
- **Enterprise Controls:** Administrators can enforce `disableAllHooks: true` in `~/.claude/settings.json`, run Claude Code with `--safe-mode`, or deploy policy mods like `sec-default` to restrict unmanaged plugin execution.

---

3. Verified Commands and Tooling

Verified Commands (from Official Documentation)

- Check installed version:
  `claude --version`
- Launch session in safe mode (disables installed mods and hooks):
  `claude --safe-mode`
- View loaded plugins and active mods inside a session:
  `/plugin`
- Install a plugin from a marketplace:
  `claude plugin install <plugin-name>@<marketplace>` *(or inside a session: `/plugin install <plugin-name>@<marketplace>`)*
- Validate plugin manifest and inspect declared hooks prior to installation:
  `claude plugin validate <plugin-dir>`
- Reload plugins within an active session:
  `/reload-plugins`
- Inspect local Desktop session status and version:
  `/status`

Unverified Reel Workflows

- Installing mods directly via bare `/plugin` without specifying repository/marketplace targets is an unverified shortcut in the reel.
- Any commands referencing external setup scripts from `ducktate.com` remain unverified community scripts.

---

4. Common Mistakes and Pitfalls

1. **Treating Mods Like Sandboxed Extensions:** Assuming a community mod cannot access filesystem secrets. Always run `claude plugin validate <plugin-dir>` before loading unfamiliar plugins.
2. **Expecting Built-in Multi-Session State:** Writing a single-session mod and expecting it to automatically see other terminal processes without reading external lockfiles or session registries.
3. **Overriding Enterprise Rules:** Attempting to bypass security defaults when `disableAllHooks` or organizational policy mods are enforced.
4. **Relying on Deprecated Flags:** Setting `CLAUDE_CODE_ENABLE_FUNCTION_HOOKS=0` (ignored as of v2.1.287).

---

5. Reusable Scaffolding Prompt

```
Create a local, read-only Claude Code mod plugin named "session-metrics".
Adhere strictly to official Claude Code mod architecture (Claude Code v2.1.287+):
1. Generate plugin.json, hooks.json, and hooks/register.js.
2. The mod must register a 'ui.render' hook to display a status banner above the prompt
showing current turn token count and tool call frequency.
3. Add a custom '/metrics' command that prints read-only session telemetry to the transcript.
4. Ensure zero external network calls, zero file writes outside ~/.claude/session-
metrics.log, and no automated approval of tool execution.
5. Output full TypeScript/JavaScript source code and validation steps using 'claude plugin
validate .'.
```

Original sources

- https://www.instagram.com/reel/DeCu7c8BZmd/